

HAMNAගේ SECURITY WORKSTREAM

Configuration, transport, dependency security, uploads, automated tests සහ CI

Sinhala practical guide - code locations, click steps සහ commands

Member: R.H.F. Hamna (IT22516916)

මෙම guide එක screenshots නැතිව හඳුලා තිබේ. එය බලන්න VS Code, browser DevTools සහ terminal එකේ ඔබේ code සහ results පෙන්විය හැකියි.

Hamnaගේ scope

V-07 error disclosure, V-08 CORS, V-09 backend headers, V-12 dependencies, V-14 secrets, V-15 HTTPS transport, V-17 upload validation, V-18 frontend headers/CSP, security tests සහ CI.

Demo folders

Original API: D:\ass_sd\demo\original\pubudu-pos-api
Secure API: D:\ass_sd\demo\secure\pubudu-pos-api-secure
Original web: D:\ass_sd\demo\original\pubudu-pos-front-end
Secure web: D:\ass_sd\demo\secure\pubudu-pos-front-end-secure

ඉක්මන් map එක

මනේ එක් එක් issue එකේ Before -> Fix -> Code -> Click steps -> Tool -> Viva sentence flow එකම භාවිතා කරනවා.

ID	Hamnaගේ security part	පිරිමාන tool
V-07	Safe error responses / no internal leaks	PHPUnit + browser Network
V-08	CORS origin allow-list	curl / PHPUnit
V-09	API response headers	curl -I / DevTools
V-12	Package advisories and upgrades	composer audit / npm audit
V-14	Secrets and .env protection	git + gitleaks CI
V-15	HTTPS API proxy transport	vercel.json review
V-17	Spreadsheet upload validation	Browser file chooser + unit tests
V-18	Frontend CSP/security headers	DevTools Network

වැදගත් සීමාව

CORS එක authentication එකක් නොවේ. Security headers එකක් පමණක් XSS fix එකක් නොවේ. `npm audit 0` කියන්නේ known package advisories එම වලොවේ නැති බව පමණයි. හැම fix එකම server-side authorization සහ safe coding සමඟ වැඩ කළ යුතුයි.

V-07 - Errors, logs සහ source disclosure

මයෙ මොකක්ද?

API එක internal SQL error, PHP file path, framework class name හෝ stack trace browser එකට යැවීමෙන් attacker කෙනෙකුට system structure එක දැනගන්න පහසු වනවා.

Original problem: Original error handling එක raw exception detail client response එකට ගෙන යා හැකි වුණා. Production template එක debug mode දක්වා යාමේ අවදානමක් තිබුණා.

Secure solution: Secure version එක generic error response පමණක් clientට දෙනවා. Technical details server log එකට යනවා. Tests මගින් SQLSTATE, vendor path සහ model name response body එකේ නොමැති බව verify කරනවා.

Code බලන තැන්

File location: Secure API: app/Exceptions/Handler.php

File location: Secure API: tests/Feature/Security/ConfigurationTest.php - V-07 tests

File location: Secure API: .env.example - APP_DEBUG=false, APP_ENV=production

වැදගත් code excerpt

```
// ConfigurationTest.php
$response = $this->actingAs($this->admin, 'sanctum')
    ->getJson('/api/reports/products');

foreach (['SQLSTATE', 'vendor\\', '.php', 'Illuminate\\'] as $leak) {
    $this->assertStringNotContainsString($leak, $response->getContent());
}
```

ඔබම click කරලා පරීක්ෂා කිරීම

1. Secure API folder එක VS Code එකෙන් open කරන්න.
2. app/Exceptions/Handler.php open කර production response handling බලන්න.
3. tests/Feature/Security/ConfigurationTest.php open කර `test\_an\_unhandled\_error\_does\_not\_leak\_internals` search කරන්න.
4. Terminal එකෙන් ConfigurationTest run කර PASS result පරීක්ෂා කරන්න.

භාවිතා කරන tools / commands

```
cd D:\ass_sd\demo\secure\pubudu-pos-api-secure
php artisan test --filter=test_an_unhandled_error_does_not_leak_internals
```

Viva එකේ කියන්න

Clientට generic error එකක් දීලා, SQLSTATE සහ framework paths වැනි technical detail server-side logs වල පමණක් තබන ලෙස error disclosure fix කලා.

V-08 - Wildcard CORS

මගේ මොකක්ද?

CORS යනු browser එකක JavaScript එකකට වෙන origin එකක API response කියවන්න අවසර දෙන policy එකකි. එය login හෝ role authorization එකේ replacement එකක් නොවේ.

Original problem: Original config එක `allowed\_origins => ['\*']` ලෙස තිබුණ නිසා ඕනෑම website origin එකක් browser මගින් API response access කිරීමට allow වීමට ඉඩ තිබුණා.

Secure solution: Secure config එක `FRONTEND\_URLS` environment variable එකෙන් approved origins පමණක් allow කරනවා. Variable එක නැති නම් empty list - wildcard fallback එකක් නැහැ. මගේ fail closed design එකකි.

Code බලන නැන්

File location: Original API: config/cors.php

File location: Secure API: config/cors.php

File location: Secure API: tests/Feature/Security/ConfigurationTest.php - V-08 tests

වැදගත් code excerpt

```
'allowed_origins' => array_values(array_filter(
    array_map('trim', explode(',', (string) env('FRONTEND_URLS', '')))
)),
'allowed_methods' => ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
'supports_credentials' => false,
```

ඔබම click කරලා පරීක්ෂා කිරීම

1. Original config/cors.php open කර `allowed\_origins` search කර `\*` තිබුණු old setting පරීක්ෂන්න.
2. Secure config/cors.php open කර allow-list code එක පරීක්ෂන්න.
3. ConfigurationTest.php හි unknown origin test සහ configured origin test පරීක්ෂන්න.
4. Terminal එකෙන් test run කර PASS පරීක්ෂන්න.

භාවිතා කරන tools / commands

```
cd D:\ass_sd\demo\secure\pubudu-pos-api-secure
php artisan test --filter=test_an_unknown_origin_is_not_granted_cors_access
php artisan test --filter=test_the_configured_frontend_origin_is_allowed
```

Viva එකේ කියන්න

CORS wildcard එකෙන් trusted frontend origins explicit allow-list එකකට මාරු කළා. Variable එක නැති වීම access permit නොකරන fail-closed behavior එකක් භාවිතා කළා.

V-09 - Backend security response headers

මයෙ මොකක්ද?

Security headers browser behavior restrict කර clickjacking, MIME sniffing, caching සහ referrer leaks වැනි risks අඩු කරන defence-in-depth layer එකකි.

Original problem: Original API responses වල consistent security-header middleware එකක් තිබුණේ නැහැ.

Secure solution: Secure API එකේ global `SecurityHeaders` middleware එක normal response, 404 සහ error response වලටත් headers යොදනවා. HSTS HTTPS request එකකදී පමණක් යවයි.

Code බලන තැන්

File location: Secure API: app/Http/Middleware/SecurityHeaders.php

File location: Secure API: app/Http/Kernel.php - global middleware registration

File location: Secure API: tests/Feature/Security/ConfigurationTest.php - V-09 tests

වැදගත් code excerpt

```
$response->headers->set('X-Content-Type-Options', 'nosniff');
$response->headers->set('X-Frame-Options', 'DENY');
$response->headers->set('Referrer-Policy', 'no-referrer');
$response->headers->set(
    'Content-Security-Policy',
    "default-src 'none'; frame-ancestors 'none'; base-uri 'none'; sandbox"
);

if ($request->secure()) {
    $response->headers->set('Strict-Transport-Security', 'max-age=31536000; includeSubDomains; preload');
}
```

ඔබම click කරලා පරීක්ෂා කිරීම

1. SecurityHeaders.php open කර each header block එක scroll කර පරීක්ෂන්න.
2. Browser එකෙන් secure API response එකක් open කරන්න හෝ DevTools Network tab open කරන්න.
3. Network request එක select කර Response Headers section එකේ `nosniff`, `DENY`, CSP, Referrer-Policy බලන්න.
4. Local HTTP server එකේ HSTS නොපෙනීම expected බව පැහැදිලි කරන්න.

භාවිතා කරන tools / commands

```
curl -I http://127.0.0.1:8000/api/login
# Windows PowerShell alternative
(Invoke-WebRequest -Method Get http://127.0.0.1:8000/api/login).Headers
```

Viva එකේ කියන්න

API එක UI එකක් නොවන නිසා CSP `default-src 'none'` ලෙස දැඩිව තැබිය හැකියි. HSTS plain HTTP localhost response එකට නොයවන්නේ RFC requirement එක නිසාය.

V-12 - Outdated dependencies

මගේ මොකක්ද?

Old framework/package versions වල published security advisories තිබිය හැකියි. Business code හරි වුණත් vulnerable dependency එකෙන් risk එකක් එන්න පුළුවන්.

Original problem: Before evidence අනුව backend Composer advisories 41ක් සහ frontend npm advisories 23ක් තිබුණා.

Secure solution: Secure project එක Laravel 12 line එකට සහ affected frontend packages update කරලා audit checks CI එකට දාලා තියනෙවා. Saved evidence අනුව audits 41 -> 0 සහ 23 -> 0.

Code බලන තැන්

File location: Secure API: composer.json and composer.lock

File location: Secure web: package.json and package-lock.json

File location: Evidence: D:/ass_sd/evidence/before and D:/ass_sd/evidence/after

වැදගත් code excerpt

```
# backend
composer audit --no-interaction

# frontend
npm audit --audit-level=high

# CI repeats both checks on push / pull request
```

ඔබම click කරලා පතේවන ක්රමය

1. Secure API folder එකෙන් composer.json open කර Laravel version බලන්න.
2. evidence/before/composer-audit.txt සහ evidence/after/composer-audit.txt compare කරන්න.
3. Secure web folder එකෙන් package.json open කරන්න.
4. npm audit සහ composer audit commands run කර output පතේවන්න.

භාවිතා කරන tools / commands

```
cd D:\ass_sd\demo\secure\pubudu-pos-api-secure
composer audit

cd D:\ass_sd\demo\secure\pubudu-pos-front-end-secure
npm audit --audit-level=high
```

Viva එකේ කියන්න

Dependency audit zero යනු application එක 100 percent secure බව නොවගේ. එය known advisories මගේ dependency tree එකේ තැනි බව පමණයි; access control සහ business logic tests වනෙම අවශ්යයි.

V-14 - Committed secrets and environment files

මයෙ මොකක්ද?

`.env` file එකකට API keys, database password හෝ production URLs තිබිය හැකියි. එය Git history එකට ගියොත් file එක delete කළත් old commit එකෙන් leak වලො ඉන්න පුළුවන්.

Original problem: Original frontend `env` tracked වීමේ අවදානම තිබුණා. Source code වල real secret නැතිම deployment configuration සහ code එක mix කරනවා.

Secure solution: Secure version එක current tree එකෙන් real `env` ඉවත් කර `env.example` පමණක් තබනවා. `gitignore`, CI tracked-file checks සහ gitleaks මගින් regression block කරනවා.

Code බලන නැන්

File location: Secure web: .gitignore

File location: Secure web: .github/workflows/security.yml - secrets job

File location: Secure API: .github/workflows/security.yml - secrets job

වැදගත් code excerpt

```
# frontend CI check
leaked=$(git ls-files | grep -E '^\.env($|\.|.)' | grep -v '\.env\.example' || true)
if [ -n "$leaked" ]; then
    echo "Environment files are tracked (V-14)"
    exit 1
fi

# gitleaks scans full Git history
```

ඔබම click කරලා පතේවන ක්රමය

1. Secure frontend `gitignore` open කර `env` rule එක පතේවන්න.
2. `.github/workflows/security.yml` open කර `Secret scan` job search කරන්න.
3. `fetch-depth: 0` සහ gitleaks step එක පතේවන්න.
4. GitHub Actions page එකේ security workflow pass වුණ run එකක් තිබේ නම් එය පතේවන්න.

භාවිතා කරන tools / commands

```
cd D:\ass_sd\demo\secure\pubudu-pos-front-end-secure
git ls-files | Select-String '^\.env($|\.|.)'
# Empty output except .env.example is the desired result.
```

Viva එකේ කියන්න

Secret එක commit කළොත් delete කිරීම පමණක් ප්රමාණවත් නොවේ. CI එක full history scan කරලා future secret leak එකක් merge වීමට පෙර අල්ලන්න උත්සාහ කරනවා.

V-15 - Plain HTTP API proxy

මගේ මොකක්ද?

Frontend HTTPS වුණත් frontend proxy එකෙන් API වනෙ යන next hop එක HTTP නම් Authorization bearer token සහ business data network එකේ plain text ලෙස යා හැකියි.

Original problem: Original production rewrite target එක `http://...` විය. API request එකේ bearer token නිවනෙ නිසා මගේ cleartext transport risk එකකි.

Secure solution: Secure Vercel rewrite target එක `https://pubudutyres.codexpress.codes/api/:path\*` ලෙස වනෙස් කළා. Browser-to-frontend සහ proxy-to-API දකෙම HTTPS විය යුතුයි.

Code බලන නැන්

File location: Original web: vercel.json

File location: Secure web: vercel.json - rewrites block

File location: Secure web: src/lib/api.js - Authorization header

වැදගත් code excerpt

```
"rewrites": [  
  {  
    "source": "/api/:path*",  
    "destination": "https://pubudutyres.codexpress.codes/api/:path*"  
  }  
]
```

ඔබම click කරලා පරීක්ෂා කිරීමය

1. Original vercel.json open කර rewrite `destination` check කරන්න.
2. Secure vercel.json open කර `https://` target එක show කරන්න.
3. src/lib/api.js open කර bearer token Authorization header එක request interceptor එකෙන් යන බව show කරන්න.
4. Explain කරන්න: token එක browserට නොපෙනෙන proxy hop එකකත් encrypt වනේන ඔබගේ.

භාවිතා කරන tools / commands

```
# code review tool  
code D:\ass_sd\demo\secure\pubudu-pos-front-end-secure\vercel.json  
# Production deployment only: check browser Network Request URL uses HTTPS.
```

Viva එකේ කියන්න

HTTPS icon එක frontend page එකට පමණක් නොව API transport path එකේ හැම hop එකකටම වැදගත්. Bearer token එක captured වුණොත් account access ලැබිය හැකියි.

V-17 - Spreadsheet upload validation

මගේ මතකය?

File picker `accept` attribute එක security control එකක් නොවේ. File rename, drag-and-drop හෝ crafted bytes මගින් parser එකට malicious/huge file එකක් යැවිය හැකියි.

Original problem: Original imports වල validation inconsistent විය: සමහර තැන් extension පමණක් check කළා, සමහර තැන් size cap එකක් තැබූ තවත් තැන් validation නැතිව parser එකට යැව්වා.

Secure solution: Secure shared validator එක file existence, empty file, 10 MB size cap, extension, MIME type, magic bytes සහ parsed row count check කරනවා. Parser patch එක V-12; input validation defence-in-depth layer එකයි.

Code බලන තැන්

File location: Secure web: `src/lib/validateUpload.js`

File location: Secure web callers: `Product/Products.jsx`, `Services/Services.jsx`, `Stock/GrnAdjust.jsx`, `GRNForm/CSVImportModal.jsx`

File location: Secure web tests: `src/lib/validateUpload.test.js` (if present)

වැදගත් code excerpt

```
export const MAX_UPLOAD_BYTES = 10 * 1024 * 1024;
export const MAX_IMPORT_ROWS = 5000;

if (file.size > maxBytes) {
  return { ok: false, error: 'File is too large.' };
}

const head = new Uint8Array(await file.slice(0, 8).arrayBuffer());
if (extension === 'xlsx' && !startsWith(head, ZIP_MAGIC)) {
  return { ok: false, error: 'Bad file signature.' };
}
```

ඔබම click කරලා පරීක්ෂා කරන්න

1. Secure frontend `src/lib/validateUpload.js` open කරන්න.
2. `MAX\_UPLOAD\_BYTES`, `ZIP\_MAGIC`, `OLE2\_MAGIC`, `validateSpreadsheetFile` search කර පරීක්ෂන්න.
3. Upload screen එකේ invalid extension, empty file හෝ too-large file එකක් තෝරලා user-friendly error toast පෙන්වන්න. Real malicious file run නොකරන්න.
4. Unit tests නිබේ නම් run කර PASS පෙන්වන්න.

භාවිතා කරන tools / commands

```
cd D:\ass_sd\demo\secure\pubudu-pos-front-end-secure
npm test
# Browser demo: use a harmless empty .txt file or an over-limit test file; do not parse untrusted malware.
```

Viva එකේ කියන්න

File extension එක පමණක් විශ්වාස කරන්නේ නැහැ. Magic bytes සහ size/row limits parser එකට යාමට පරෙ enforce කරන නිසා renamed binary සහ resource-exhaustion risk අඩු වෙනවා.

V-18 - Frontend CSP සහ response headers

මයෙ මොකක්ද?

Frontend SPA එකට API JSON policy එකට වඩා වෙනස් policy එකක් අවශ්‍යයි, මන්ද scripts, fonts, images සහ approved API/Google connections අවශ්‍යයි.

Original problem: Original frontend deployment configuration එකේ headers block එකක් සහ CSP එකක් නොතිබුණා.

Secure solution: Secure `vercel.json` එක frontend-specific CSP, HSTS, nosniff, frame/referrer/permissions policies යොදනවා. CSP එක stored XSS fix එකට defence-in-depth එකක් පමණයි; unsafe HTML output escape කිරීම තවම අනිවාර්යයි.

Code බලන තැන්

File location: Secure web: vercel.json - headers block

File location: Secure web: vercel.json - Content-Security-Policy

File location: Secure web: .github/workflows/security.yml - security lint

වැදගත් code excerpt

```
{
  "key": "Content-Security-Policy",
  "value": "default-src 'self'; script-src 'self'; ...; object-src 'none'; frame-ancestors 'none'; upgrade-insecure-requests"
}

{ "key": "X-Frame-Options", "value": "DENY" }
{ "key": "X-Content-Type-Options", "value": "nosniff" }
```

ඔබම click කරලා පරීක්ෂා කිරීම

1. Secure web vercel.json open කර headers array පරීක්ෂන්න.
2. `connect-src` නුළු approved API සහ Google origins ඒතරක් තිබෙන බව පරීක්ෂන්න.
3. Deployed site එකේ DevTools > Network > document request > Response Headers open කරන්න.
4. Local Vite server එකේ Vercel headers නොපෙනිය හැකි බවත් production deployment එකේ verify කළ යුතු බවත් කියන්න.

භාවිතා කරන tools / commands

```
# CI security lint and build
cd D:\ass_sd\demo\secure\pubudu-pos-front-end-secure
npm run lint:security
npm run build
```

Viva එකේ කියන්න

Frontend CSP එක external script/data exfiltration attempts අඩු කරන defence-in-depth layer එකකි. එය application-level output escaping වෙනුවට භාවිතා කළ නොහැක.

Security test suite සහ CI gate

Hamnaගේ integration contribution

Fix එකක් ලිව්වා පමණක් ඒරමාණවත් නැහැ. Security regression test එකක් සහ CI gate එකක් තිබුණොත් future commit එකකින් old bug එක නැවත ආවත් merge කිරීමට පෙර catch කිරීමට දත්සාහ කරයි.

Backend files

File location: tests/Feature/Security/ConfigurationTest.php - V-07/V-08/V-09

File location: tests/Feature/Security/AccessControlTest.php - V-01/V-02/V-03

File location: tests/Feature/Security/AuthenticationTest.php - V-05/V-06

File location: tests/Feature/Security/BusinessIntegrityTest.php - V-04/V-10/V-11

File location: .github/workflows/security.yml - PHP tests, Composer audit, secret scan

Frontend files

File location: .github/workflows/security.yml - security lint, unit tests, production build, npm audit, secrets

File location: eslint.security.config.js - dangerous browser patterns

```
# Backend security suite
cd D:\ass_sd\demo\secure\pubudu-pos-api-secure
php artisan test --testsuite=Security
composer audit

# Frontend checks
cd D:\ass_sd\demo\secure\pubudu-pos-front-end-secure
npm run lint:security
npm test
npm run build
npm audit --audit-level=high
```

GitHub Actions click steps

1. GitHub repository open කරන්න.
2. Actions tab click කරන්න.
3. Security workflow select කරන්න.
4. Latest successful run open කර jobs බලන්න: tests, dependencies, secrets.
5. Green check තිබේ නම් පමණක් CI passed ලෙස කියන්න.

Important wording

`0/14 exploitable` කියන්නේ 14 attack attempts අතරින් successful exploit ගණන 0 යන්නයි. එය 14 tests fail වුණා කියන එක නෙෙව්වේ. Full lint errors තිබීම security lint failure එකක් ලෙස කියන්නත් එපා; security lint වනේවි run වනෙවා.

Hamnaගේ viva checklist

1. මුලින් scope එක කියන්න: configuration, transport, dependency hygiene, upload validation, tests සහ CI.
2. V-08 ගැන අහුවොත්: CORS authentication එකක් නොව browser origin policy එකක් කියන්න.
3. V-09 ගැන අහුවොත්: headers defence-in-depth; HSTS HTTP localhost එකේ නැතිවීම correct behavior කියන්න.
4. V-12 ගැන අහුවොත්: audit zero = known dependency advisories zero; all bugs zero නොවෙත් කියන්න.
5. V-14 ගැන අහුවොත්: delete කරලා පමණක් නොව CI + gitleaks full history scan කියන්න.
6. V-15 ගැන අහුවොත්: browser HTTPS පමණක් නොව proxy-to-API hop එකත් HTTPS විය යුතුයි කියන්න.
7. V-17 ගැන අහුවොත්: extension/MIME පමණක් trust නොකර size + magic bytes + row limit ද check කරන බව කියන්න.
8. අවසානයේ CI run එකෙන් security regression merge වීමට පරෙ catch කරන බව කියන්න.

Short final statement

Hamnaගේ work එකේ අරමුණ වූයේ code-level fixes පමණක් නොව application එක වටා ඇති security configuration, package supply chain, transport path, browser policy සහ future regression controls එකට සම්බන්ධ කිරීමයි.